

ERD and Relational Schema: Database Design

Patrick Tilotta, Santiago Yuriar, Dawson Merriman, Colby Christy

Step 1: Provide Business Rules and Constraints

1. User and Wishlist

Business Rule: A user may have zero, one, or many wishlist entries, but each wishlist entry must belong to one and only one user.

This rule reflects that a user is not required to add games to their wishlist, so the relationship is optional for a user. However, a wishlist entry cannot exist without being associated with a user, so the relationship is mandatory for Wishlist.

Constraint

- A wishlist entry must contain a valid UserID
- A user can exist without any wishlist entries

2. Game and Wishlist

Business Rule: A game may appear in zero, one, or many wishlist entries, but each wishlist entry must refer to one and only one game.

A game may never be added to any user's wishlist, so participation is optional for Game. Also, a wishlist entry cannot exist unless it refers to a specific game, so the relationship is mandatory for Wishlist.

Constraint

- A wishlist entry must contain a valid GameID
- A game can exist without appearing in any wishlist

3. User and Game through Wishlist

Business Rule: A user may wishlist many games, and a game may be wishlisted by many users.

This creates a many-to-many relationship between User and Game. Because many-to-many relationships cannot be directly implemented in a relational database, the relationship is resolved through the Wishlist relationship entity.

Constraint

- Each wishlist record connects one user to one game
- The same user should not have duplicate wishlist entries for the same game

4. Developer and Game

Business Rule: A developer may develop zero, one, or many games, but each game must be developed by one and only one developer.

A developer may exist in the system before any games are associated with them, so participation is optional for developers. Which is why every game in the system must have exactly one developer, so participation is mandatory for Game.

Constraint

- Each game must contain a valid DeveloperID
- A developer can exist without any games

5. Game and GamePlatform

Business Rule: A game may be available on zero, one, or many platforms, but each GamePlatform record must refer to exactly one game. A game can exist in the system even if it is not yet released on any platform. However, a GamePlatform record cannot exist without a valid game.

Constraint:

- GameID in GamePlatform is required.
- A game can exist without being linked to any platforms.

6. Platform and GamePlatform

Business Rule: A platform may support zero, one, or many games, but each GamePlatform record must refer to exactly one platform. A platform can exist in the system before any games are released for it. However, a GamePlatform record cannot exist without a valid platform.

Constraint:

- PlatformID in GamePlatform is required.
- A platform can exist without any games linked to it.

7. Game and Platform

Business Rule: A game may be available on one or many platforms, and a platform may support zero, one, or many games.

A single game may be released on multiple platforms such as PC, Xbox, or PlayStation. At the same time, each platform can support many games. This creates a many-to-many relationship between Game and Platform.

Constraint

- A game can be linked to multiple platforms
- A platform can be linked to multiple games
- Each GamePlatform record must reference:
 - one valid GameID
 - one valid PlatformID

8. Referential Integrity

Business Rule: All foreign key values must reference existing valid records in their parent tables.

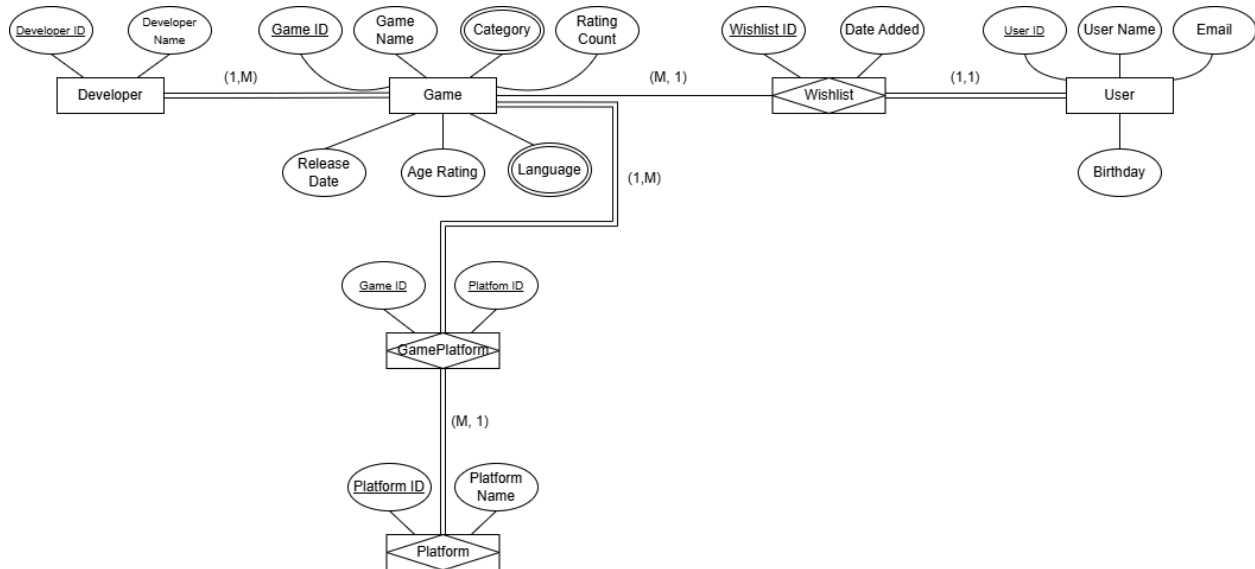
This prevents invalid or orphaned records from being stored in the database.

Constraint

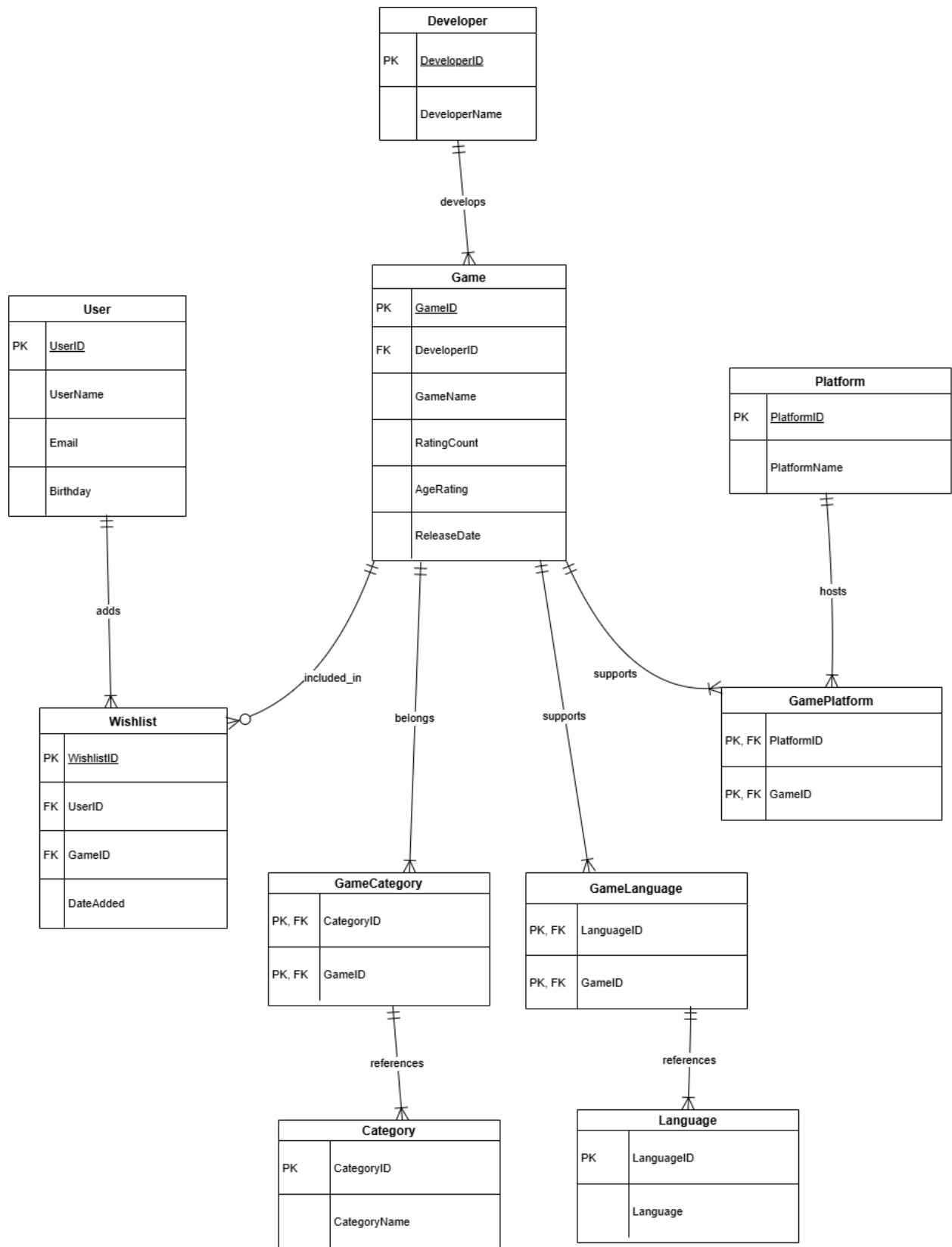
The database must enforce the following:

- Wishlist.UserID must match an existing User.UserID
- Wishlist.GameID must match an existing Game.GameID
- Game.DeveloperID must match an existing Developer.DeveloperID
- GamePlatform.GameID must match an existing Game.GameID
- GamePlatform.PlatformID must match an existing Platform.PlatformID

Step 2: Create the Entity-Relationship Diagram (ERD)



Step 3: Design the Relational Schema



Step 4: Normalize the Schema

This schema is in 3NF because it satisfies all normalization requirements. It meets 1NF since all attributes are atomic by eliminating repeating groups. Categories, languages, and platforms are separated into their own respective tables. It also satisfies 2NF because every non-key attribute is fully dependent on the primary key. For example, GameLanguage contains only GameID and LanguageID, which together identify one game-language relationship. There is no extra attribute like LanguageName or GameName stored there that would depend on only one side of the key. This also applies for GameCategory and GamePlatform, which means these tables do not contain partial dependencies. It meets 3NF because there are no transitive dependencies, and non-key attributes depend only on their primary keys. Values such as category, language, and platform are not stored directly in the Game table. Instead, we separated them into their own tables and made them link through associative tables. This prevents indirect dependencies such as storing CategoryName, LanguageName, or PlatformName repeatedly inside Game.